

High Performance Computing

Lab 3: Parallel Program Design and Monte Carlo Simulation

The purpose of this lab is to practice the basic principles of parallel program design using a simple computational problem. You will implement a Monte Carlo approximation of π , divide the computation into independent tasks, combine partial results using a reduction operation, and analyze the problem in terms of partitioning, communication, agglomeration, and mapping.

No parallel programming library is required for this lab.

Part 1: Monte Carlo Approximation of π

Consider a square with side length 2 and a circle of radius 1 inside the square.

Random points (x, y) are generated inside the square:

$$-1 \leq x \leq 1, \quad -1 \leq y \leq 1.$$

A point is inside the circle if

$$x^2 + y^2 \leq 1.$$

Let

$$N$$

be the total number of generated points and let

$$M$$

be the number of points that fall inside the circle.

The value of π can be approximated as

$$\pi \approx 4 \frac{M}{N}.$$

Write a C++ program that generates random points and calculates an approximation of π .

You may start with the following code:

```
1 #include <iostream>
2 #include <random>
3
4 int main() {
5
6     const long long N = 1000000;
7
8     std::mt19937 generator(42);
9     std::uniform_real_distribution<double> distribution(-1.0, 1.0);
10
11     long long inside = 0;
12
13     for (long long i = 0; i < N; ++i) {
```

```

14
15     double x = distribution(generator);
16     double y = distribution(generator);
17
18     // Check whether the point is inside the circle
19
20 }
21
22 // Calculate pi
23
24 return 0;
25 }

```

Your program should print:

- the total number of generated points;
- the number of points inside the circle;
- the approximated value of π .

Part 2: Effect of Problem Size

Run the program using the following values:

$$N = 10^3, \quad N = 10^4, \quad N = 10^5, \quad N = 10^6.$$

Record the approximated value of π .

N	Approximation of π
10^3	
10^4	
10^5	
10^6	

Answer the following questions.

1. Does the approximation generally improve when N increases?
2. Does the program have to store all generated points?
3. Can one generated point be processed without knowing the result for another point?

Part 3: Partitioning

Suppose that four processors are available:

$$P_0, \quad P_1, \quad P_2, \quad P_3.$$

Assume

$$N = 1,000,000.$$

Divide the generated points equally among the four processors.
Complete the following table.

Processor	Number of Points	Local Result
P_0		M_0
P_1		M_1
P_2		M_2
P_3		M_3

Answer:

1. How many points should each processor generate?
2. Can the processors perform their work independently?
3. Is this problem an example of domain decomposition or functional decomposition?
4. Does one processor need the coordinates generated by another processor?

Part 4: Simulating Multiple Processors

Modify your program so that it simulates four processors.

The program should calculate a separate local count for each processor.

For example:

```

1  const int P = 4;
2
3  long long local_inside[P] = {0};
4
5  for (int p = 0; p < P; ++p) {
6
7      for (long long i = 0; i < N / P; ++i) {
8
9          double x = distribution(generator);
10         double y = distribution(generator);
11
12         if (x * x + y * y <= 1.0) {
13             local_inside[p]++;
14         }
15     }
16 }
```

Print the local result of every simulated processor:

```

Processor 0: ...
Processor 1: ...
Processor 2: ...
Processor 3: ...
```

Note that this program is still executed serially. The purpose is to represent how the problem could be divided before introducing an actual parallel programming library.

Part 5: Reduction

After all processors finish their local computations, the total number of points inside the circle must be calculated as

$$M = M_0 + M_1 + M_2 + M_3.$$

Add this reduction operation to your program.

For example:

```

1 long long total_inside = 0;
2
3 for (int p = 0; p < P; ++p) {
4     total_inside += local_inside[p];
5 }

```

Then calculate

$$\pi \approx 4 \frac{M}{N}.$$

Answer the following questions.

1. Is calculating each M_p an independent computation?
2. Is calculating the final M an independent computation?
3. What information must processors communicate at the end?
4. Is the final summation an example of local communication or global communication?
5. What is this operation called in parallel computing?

Part 6: Agglomeration

Suppose we initially create one task for every generated point.

For

$$N = 1,000,000,$$

this would create one million very small tasks.

Instead, we can combine many points into larger tasks.

For example, one task could process

$$10,000$$

points.

Answer:

1. How many tasks would be created if each task processes 10,000 points?
2. Is one task per point an example of fine-grained or coarse-grained parallelism?
3. Why can combining many small tasks into larger tasks reduce overhead?
4. What disadvantage could occur if the tasks become too large?

Part 7: Mapping

Assume that eight equal tasks are created:

$$T_0, T_1, T_2, T_3, T_4, T_5, T_6, T_7.$$

There are four processors:

$$P_0, P_1, P_2, P_3.$$

Assign the tasks so that each processor receives approximately the same amount of work. Complete the table.

Processor	Assigned Tasks
P_0	
P_1	
P_2	
P_3	

Answer:

1. How many tasks does each processor receive?
2. Is the workload balanced?
3. What happens to the total execution time if one processor receives much more work than the others?

Part 8: Parallel Program Design

For the Monte Carlo problem, briefly describe each of the following steps.

1. Partitioning:

What should be divided into independent tasks?

2. Communication:

What information needs to be communicated between tasks?

3. Agglomeration:

Why could many individual point-generation tasks be combined into larger tasks?

4. Mapping:

How should these tasks be assigned to available processors?

Part 9: Short Analysis

Consider the main computation:

```

1 for (long long i = 0; i < N; ++i) {
2
3     double x = distribution(generator);
4     double y = distribution(generator);
5
6     if (x * x + y * y <= 1.0) {
7         inside++;
8     }
9 }
```

Answer the following questions.

1. What is the time complexity of this loop in Big-O notation?

2. If N is doubled, approximately how much additional work must be performed?
3. Which part of the algorithm can be executed independently by different processors?
4. Which operation requires the results from all processors?

Submission

Submit:

1. the C++ source code;
2. the results for different values of N ;
3. the output showing the local results of four simulated processors;
4. the final approximation of π after reduction;
5. completed tables and answers to all questions.